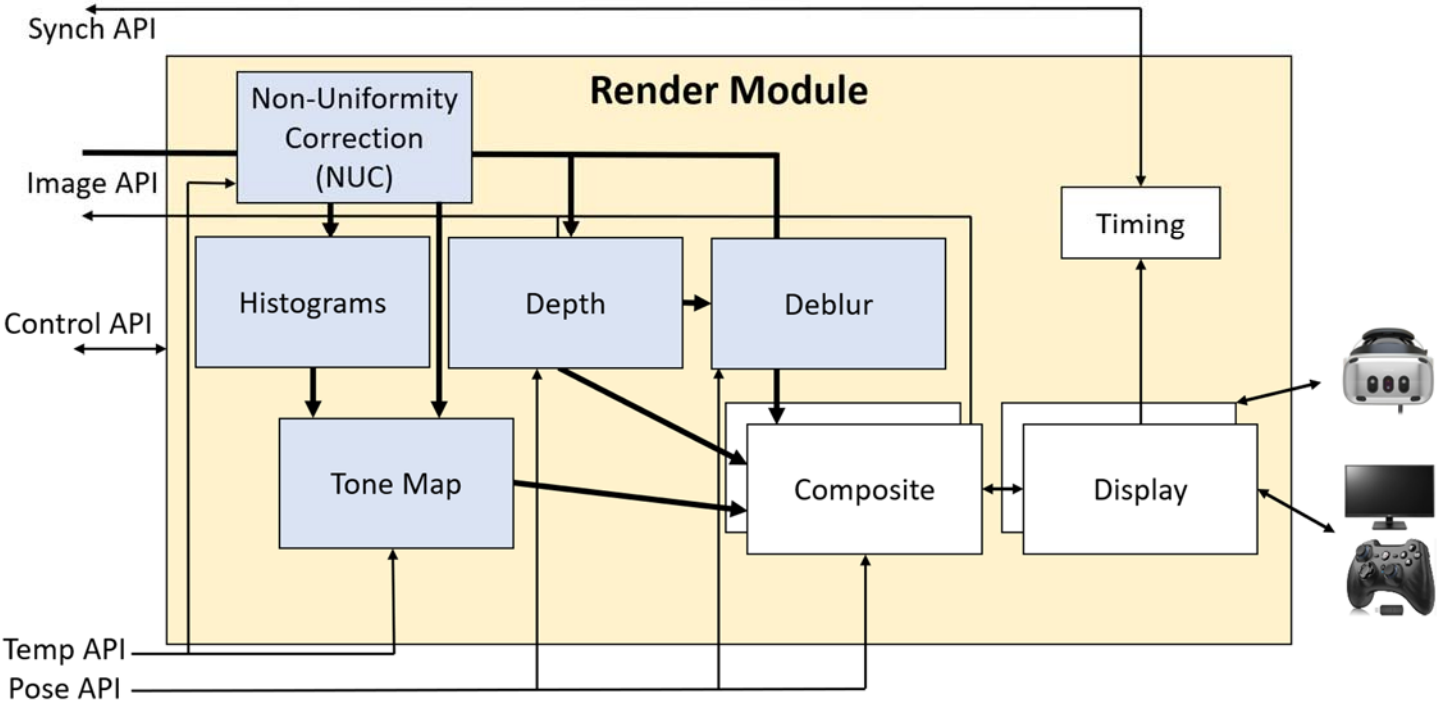
Render Implementation

This document describes the implementation design for the Render Module for the Apache IR strap-down pilotage project. It is based on the architecture described in the *TR-006v29_Software_Architecture*, *TR-008v03_Advanced_Features_Cost_Estimates*, and *TR-009_v08_Core_API_Implementations* documents along with decisions made through 10/29/2025.

Render Module architecture



The diagram above, from *TR-006v04_Software_Architecture*, includes Phase-I and Phase-II AI components. The system can operate with infrared cameras (IR) or with monochromatic visible-light cameras (VIS).

Design decisions

- **Thread per camera ingest:** Tests described in *TR-009_v07_Core_API_Implementations* showed the need to have a thread per camera to reliably ingest at full rate without missing packets.
- **Two displays:** Ability to drive at least one wide-FOV and one narrow-FOV display from the same Render Module.
- **Cross platform:** When available, cross-platform (Windows/Linux, i86/ARM) libraries and approaches will be used.

Hardware

- **Intel/AMD 64-bit CPUs with 24+ cores (32+ hyperthreads) and 32GB+ RAM.**
- **nVidia GeForce 4090 (24GB GPU RAM):** This is the baseline GPU to be used for implementation.
- **Point-to-point 40 or 100 gigabit fiber Ethernet:** Connection to one or more Core Modules for live and replay through external Core and to an external NAT drive storing data for client-side Core Module implementation.
- **Displays:** The displays to be supported by the system include the following:
  - Elbit Xsight HMD (50 Hz).
  - HD or 4K external display showing 1280x1024-pixel image with 40x30-degree FOV (60 Hz).
  - 75” Samsung 8K display for wide-FOV (60 Hz).
  - Varjo XR-4 HMD (90 Hz).

- **User interaction device:** Hand-held game controller with wired connection for monitors, built in tracking on Xsight and XR-4.

Software

- **CMake:**[1] This cross-platform build system will be used to configure and build.
- **CUDA:**[2] Image processing will be accelerated using the nVidia CUDA library.
- **Display Libraries:** See *Appendix A* for a discussion of the available window-creation and display libraries.
    - **GLFW:** This window-creation library will be used for the flat-screen monitor configurations and it will be the initial implementation for the Elbit HMD. It will be the first Display submodule implementation and will support rendering on both Windows and Linux (Intel and ARM).
    - **OpenXR:** The Display Submodule for the Varjo display will use OpenXR to read desired poses and perform latency hiding. It will be the second Display Submodule implementation.
    - **(if needed) OSVR:** If the full-screen display latency is too high for the Elbit display or other monitors, OSVR will be used, along with its VRWorks Direct Mode interface or a new Windows.Devices.Display.Core interface.
- **OpenGL:**[3] This cross-platform graphics language will be used. It is supported on the Varjo OpenXR runtime and it supports CUDA interoperability and all of the rendering tools required.
- **VRPN:**[4] The Virtual-Reality Peripheral Network cross-platform VR device interface will be used to connect to the user-interaction device, using a server run by the Render Module as a separate thread with a locally-connected client using callback responses rather than network loopback.
- **Operating systems:**
    - **Windows 10 or 11 Pro:** The Varjo HMD requires Windows 10 or 11 to operate. Many commercial HMDs and VR libraries only run on Windows. The initial implementation will use this operating system.
    - **Ubuntu Linux:** The availability of non-compositing windowing libraries may make it easier to develop low-latency rendering on Linux than on Windows for non-VR displays. The Render Module with the OpenXR Graphics Helper Display Submodule will be built and maintained on Linux as well as Windows.
- **C++:**[5] The Render Module application and its submodules will be developed in C++.

Protocols

- **JSON:**[6] JavaScript Object Notation will be used for configuration data.

[1] https://cmake.org
[2] https://developer.nvidia.com/cuda-toolkit
[3] https://www.opengl.org
[4] https://vrpn.net
[5] https://en.wikipedia.org/wiki/C%2B%2B
[6] https://www.json.org

Render Module implementation

The Render Module will run as a stand-alone executable. An *Advanced Installer*[7] package will be provided to install it on Windows. A CMake "make install" target will be provided to install it on Linux.

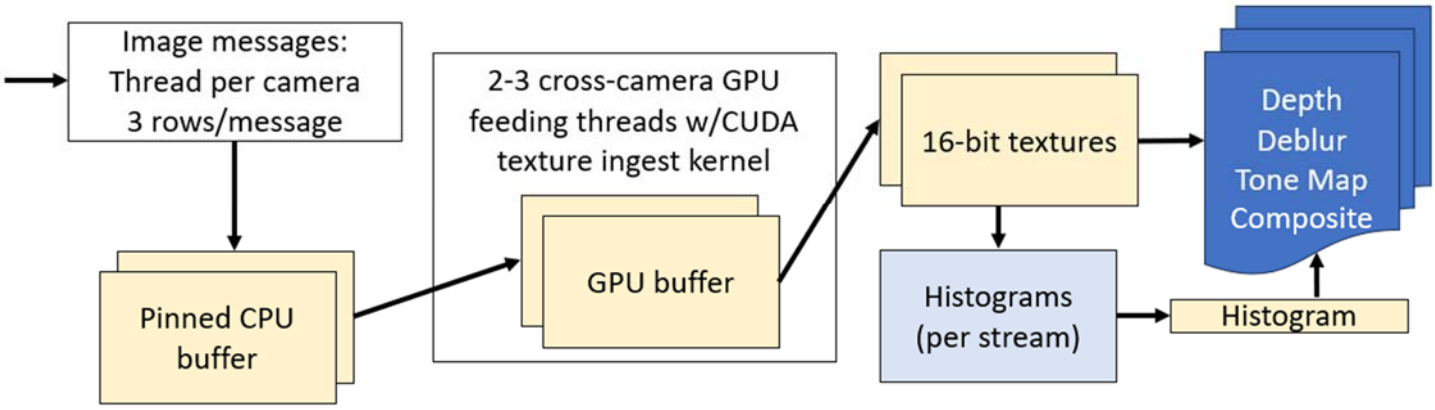
The cadence of the Render Module is driven by the *Display* submodule, which calls the *Composite* submodule. The others are driven by data ingestion. Ingestion is described first, followed by the submodules.

Ingestion

For optimal performance, ingestion has the following characteristics:

- **Sorting:** The required use of multiple threads to ingest the video from all cameras sometimes results in out-of-order packet delivery to the camera ingest threads (much less than a frame time). A sorted queue is maintained by each camera to ensure that they are in proper order when processed.
- **Streaming:** Analysis in *TR-001v04_System_Latencies* showed the need to stream image data all the way to the GPU buffers without intermediate full-image buffering to minimize latencies.
- **Overlapped:** Tests described in *TR-007v01_IR_Camera_Color_Calibration* showed the benefit of overlapped pixel data transmission and histogram or non-uniformity calculations to reduce latency. Down-stream calculations (tone mapping, depth estimation, deblurring and compositing) cannot be overlapped with transmission because they require the histogram and/or entire image.
- **Parallel:** Tests described in *TR-009_v01_Core_API_Implementations* showed the need to have a thread per camera to reliably ingest at full rate without missing packets. There must also be 2-3 threads to copy data from CPU data to GPU textures (Linux requires 2 for 21 cameras and 3 for 25; Windows requires 2 for each).
- **Buffer pools:** To avoid the overhead of allocating CPU pinned-memory and GPU buffers for each frame, pools of buffers are pre-allocated and then reused when processing is completed on them.
- **Pre registered resources:** To avoid the need to register and unregister OpenGL textures for CUDA kernels to copy to, a mapping from texture ID to cudaGraphicsSource is maintained. Each texture is registered only once.

To achieve this, CUDA streams with asynchronous overlapping kernel execution with data transfer on other streams is used.



Data to pinned CPU buffers: There is a thread per camera reading network data and copying the data from each packet into a standard CPU buffer and then into a pool of pinned CPU buffers associated with the camera. A thread-safe queue is used to send these requests with this data from each camera to one of a set of threads that sends them to the GPU.

[7] https://www.advancedinstaller.com

Data to OpenGL textures: A set of CPU threads has a CUDA stream per camera to perform an asynchronous copy from pinned memory followed by one required and one optional kernel operation on the copied data (copy-to-texture kernel, optional NUC kernel that will apply per-pixel offsets and gains, and optional histogram calculation kernel) that produce 16-bit monochromatic OpenGL textures. It makes sure to only process a minimum number of rows at a time (see note below). It always takes all available rows (more than one image message may have been delivered since its last run). This writes to a texture that is not being used for rendering (four textures are provided per camera – up to two for the current render frame, one for filling, and one lying fallow to handle synchronization issues). The most-recently-completed set of textures across all cameras is always used for rendering.

Note: The *Time_CUDA_write.cu* test program was used to test the speed of sending N lines at a time from 25 and 21 images, round-robin among the cameras, using a different CUDA stream for each camera. It was run on an Alienware M18 R1 laptop with an nVidia GeForce 4080. When sending 8 lines at a time from 21 cameras, it was able to reach 62fps. When sending 16 lines at a time from 25 cameras, it was able to reach 79fps (32 lines, 120fps; 4 lines, 33fps; 2 lines, 18fps, 1 line 9fps). (Similar speeds were achieved when copying all sections within the same camera then moving to the next camera.) (Using multiple threads to perform the copies reduced performance; however, using a single separate thread from the main thread sped it up sometimes.) This indicates that we should ensure that the transfers wait until we have ***8-16 lines available (for 21-25 cameras)*** to reduce per-copy overhead and that ***a small number of copy threads is optimal***. Final testing with the full Render Module code indicated that 16 lines is sufficient for Linux but that Windows requires more – 110 packets (330 lines) was chosen for 21 cameras and 330 packets (990 lines) for 25. For Linux, we use 2 threads for 21 cameras and 3 for 25; for Windows we use 2 threads for both conditions.

Further testing revealed that we can get fully overlapping communication and kernel operation when sending the entire image, or half of the image, and performing 300 or more multiplies and divisions per pixel on the resulting data. As the number of lines sent is reduced, interleaving transmission and computation causes gaps between the operations, slowing the overall throughput. For ***25 cameras***, we could get 81fps when sending and computing on ***32 lines at a time***. For 21 cameras, we could get 65fps when sending and computing on 16 lines at a time. 32 lines is about half a millisecond at 60fps, so we should incur only a small amount of latency by sending multiple lines.

As of 10/2/2024, the render system has been shown to operate with 21-25 incoming cameras, streamed either live from a remote simulated camera or from storage.

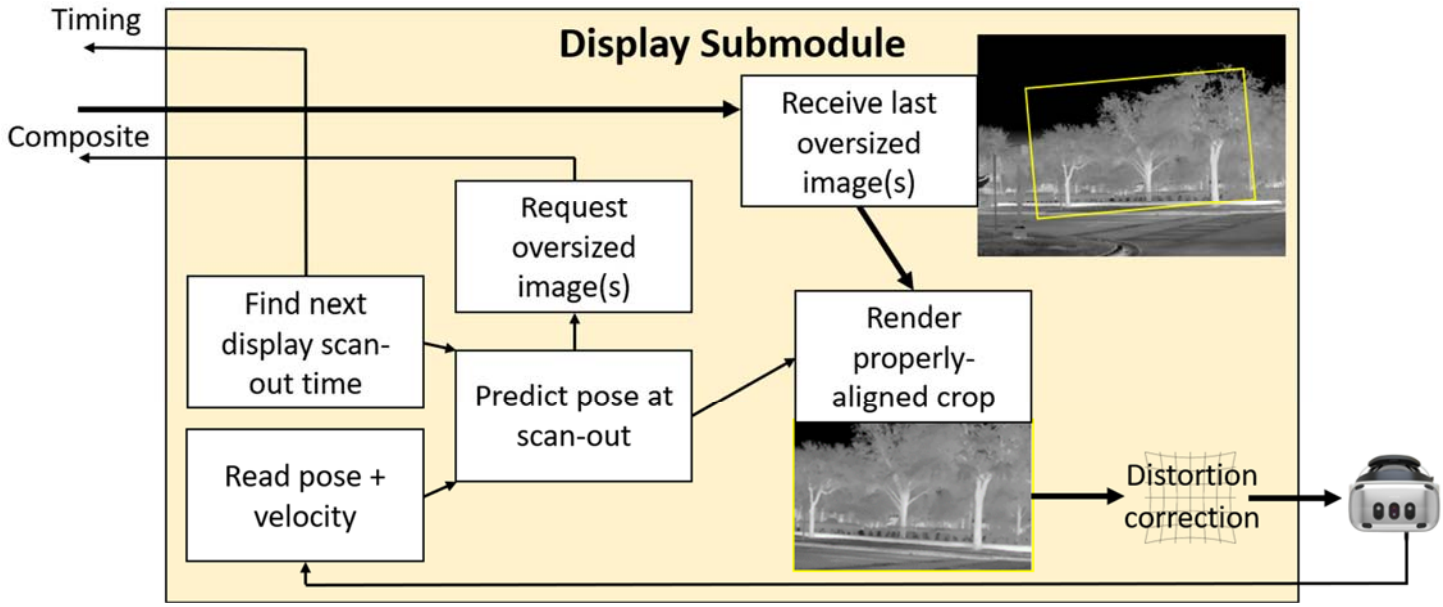
Timing submodule

Incoming *Clock Sync* messages are used by this submodule to estimate the offset and scale between the Core Module clock and the local system clock. It is initialized once the first message has been received, setting the offset based on its reading and setting the scale to 1.

As each group of 100 messages is received, the one with the minimum clock offset will be stored along with its local time of arrival minus the expected transmission time. A list of the 100 most-recent minima (each from a set of 100) will be maintained, with the oldest dropping off the list when it overflows. Once this list has at least 10 entries, a line will be fit to the entire list and its offset and scale used as the estimate for the time difference between the two clocks.

A method will be provided to the other modules to adjust a local-time software synchronization control packet to Core Module time and send it.

Display submodule



The diagram above, from *TR-006v04_Software_Architecture*, shows the operations performed by the Display submodule. This module uses one of the selected *Display Libraries* (along with user input from *VRPN* if needed) to determine display timing, perform prediction, request image and display (warped, cropped, and undistorted) images from the Composite submodule. The implementation differs considerably between libraries, so each is treated separately below.

When two Display submodules are operating, at most one of them will send synchronization command packets. Each will have its own OpenGL context (with shared textures and CUDA data among them).

Window/VRPN

To provide smooth display, a dedicated thread that busy-waits on its operations will drive this submodule.

Monitor: Viewing **pose and velocity** will be driven by a 2-axis joystick read using VRPN. Each axis has an analog input in the range [-1,1] that will be mapped to a pan and tilt velocity, which will be numerically integrated and then clamped to keep the viewport center from leaving the field of regard of the cameras. The most-recent reading provides the instantaneous velocity estimate. The view "up" direction will always remain vertical as the viewpoint rotates around the origin. **XSight:** View pose will be read from the Ethernet interface.

The **next display scan-out time** will be determined by adding the display period to the time read immediately after the previous frame's swap-buffers command. The window-construction library will be configured to block until vsync has completed. A low-pass filter will be applied to the vsync times to avoid introducing offsets from OS scheduling glitches. This scan-out time will be used to send a **synchronization command packet** after conversion to Core module time.

The **predicted pose at scan-out** will be estimated by adding the current velocity multiplied by the time until the middle of the next frame to the current pose estimate and then clipping to the field of regard. The viewport (and its pixel count) will be expanded by a configurable amount (perhaps 5 degrees) to provide a buffer in case of prediction error.

The submodule will wait until a configuration-defined delay ahead of the next vsync before **requesting a new render** from the Composite submodule. This can reduce the end-to-end latency by ensuring that the most-recent consistent set of frames from all cameras will just have completed processing before rendering is requested. The OpenGL buffer to be written into will have been allocated by the Display submodule whose resolution is slightly larger than the resulting monitor to allow for full resolution when the slightly smaller viewing region is displayed. The origin-centered viewpoint(s) used to render are specified as parameters to the *Render()* call.

A new pose estimate for the viewpoint at the time halfway through presentation of the upcoming image will be computed. Together with the information from the Composite submodule, it will be used to construct a viewing transform that **properly aligns and crops** the most-recent frame, which will be rendered waiting for vsync into a full-screen, undecorated window whose resolution matches the monitor it is displayed on.

For driving flat-screen monitors and the Xsight HMD, **no distortion correction** is required[8]; it will not be implemented in this version.

OpenXR

The Varjo display will be driven by OpenXR, which internally handles all operations except sending the Timing synchronization signal. The time of the next vertical synchronization can be queried from the library and used to send a **synchronization command packet** after conversion to Core module time when using 60Hz HMD or monitor displays.

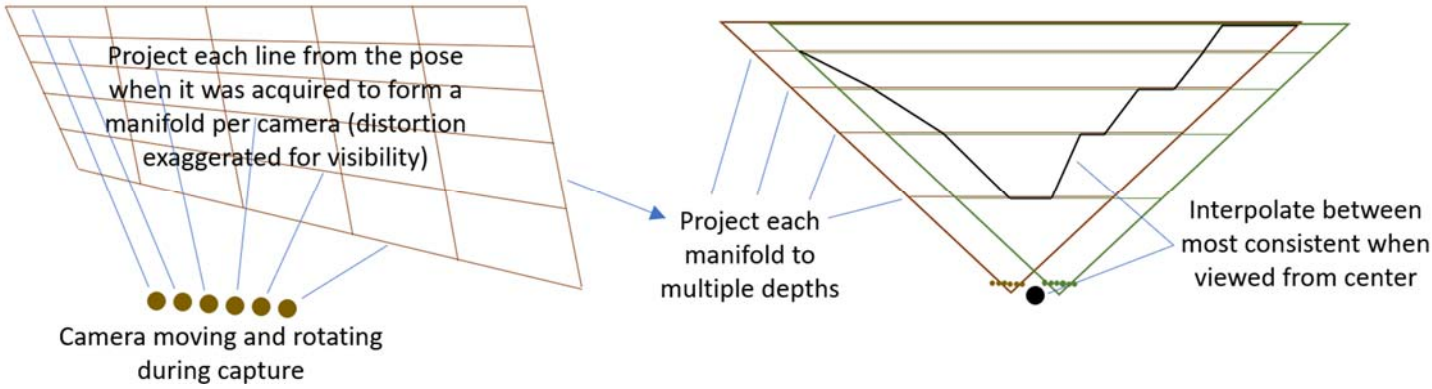
Note: The Varjo HMD has an update rate of 90Hz, which is faster than the camera capture rate of 60Hz. This will cause the same frame to be displayed twice for every other camera frame. The motion of the scene and viewer during this time will be accounted for, so motion will appear to be smooth. However, frame-to-frame noise will stutter.

OpenXR also provides a method to request high-resolution "foveal" images to reduce the total rendering work by only rendering at high resolution near the centers of the displays. These could be used to drive down-sampled displays from the Composite submodule, but initially we will render all portions at full video resolution.

Depth submodule (optional)

There are two wide-FOV cameras with overlapping fields of view for each half of the field of regard. The basic approach is to find (for each region in the image) the depth at which the images from the two cameras best agree and then to interpolate between region centers to produce a 2D surface in 3D that estimates the manifold to project high-resolution textures from the narrow-FOV cameras onto.

TR-008v03_Advanced_Features_Cost_Estimates outlines the approach to be taken for depth estimation, which is depicted in the figure and described below.



As seen on the left side of the figure, progressive scan-out causes each camera's pose to move and rotate while the image is acquired, causing each scan line to be translated and rotated compared to the others. The procedure to handle this is as follows:

- Determine the average location and rotation of the two cameras over the course of scan-out.
- For each camera (one camera is shown in the image, drawn in brown): At evenly spaced pixels in the horizontal and vertical directions, determine the vector through the pixel from the camera center of projection at the time

[8] The lack of distortion correction has not been confirmed on the Xsight. Its documentation does not mention it, and it is presumed to be handled inside the device itself.

the pixel was scanned (including the calibrated lens distortion). The origin of the vector is the camera center of project. The length is sufficient to project onto a plane 1 meter in front of the camera center of projection.

The right side of the figure shows how the unit manifolds from the brown and green camera are combined to determine the projection manifold:

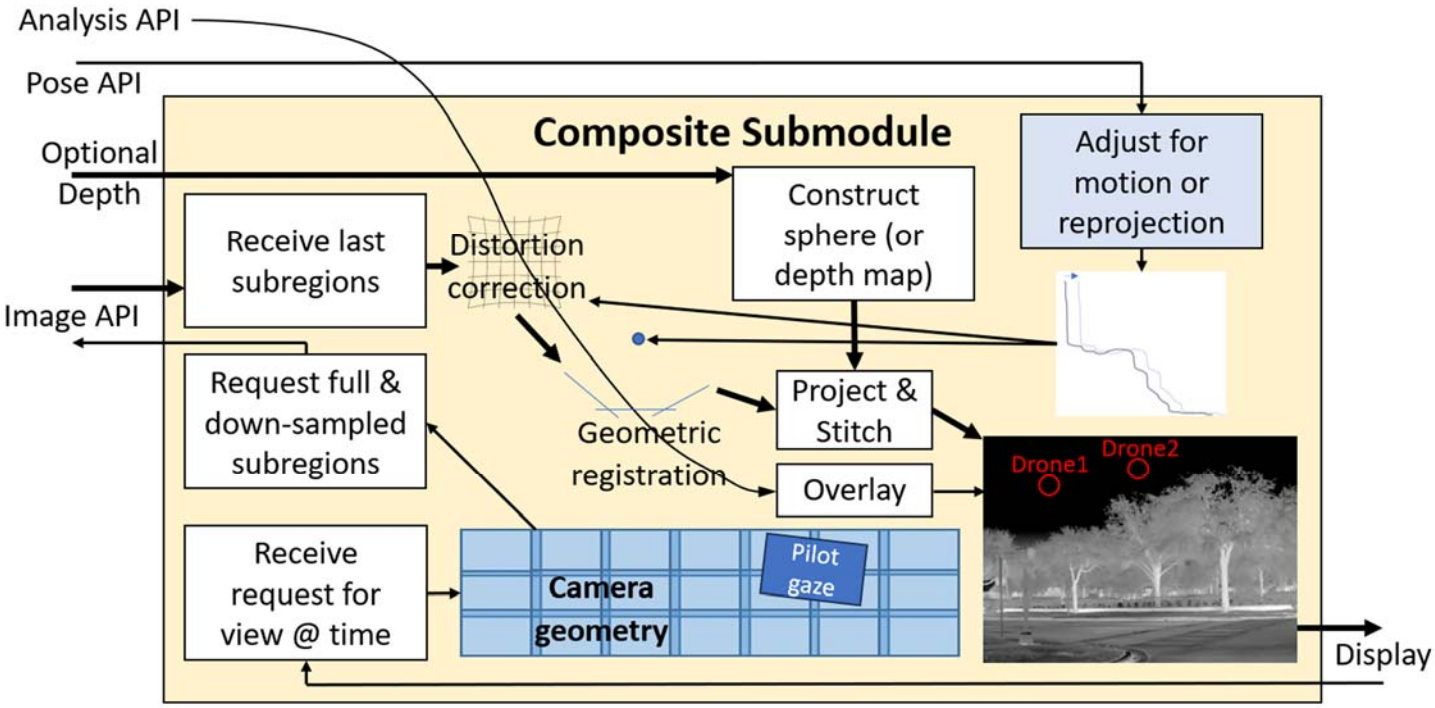
- Select a set of depths (perhaps 5, 7, 10, 14, 19, 25 meters, and 1km) to sample. At each depth:
  - Project the points from the unit manifold from each camera (shown in brown and green) to the selected distance, combining each into a rectangular mesh.
  - Map the image from each camera onto the mesh and draw it from the average location determined above. This produces two images, one from each camera, at the selected depth.
  - For regularly-spaced regions in X and Y in the rendered image, compute the sum of the pixelwise squared difference: (brown pixel – green pixel) * (brown pixel – green pixel).
- For each region, keep track of the depth with the minimum sum of squared differences. When the selected depth matches the actual depth of the object, these differences are expected to be smallest – the pixels from the two images will align with each other. The minimum sum should indicate the best depth for the region.
- Construct the manifold by joining the centers of the minimum-error depth for the region at each grid location, projected to the proper distance from the average location.

The resulting pair of manifolds (2D surfaces in 3D) from each pair of wide-FOV cameras provides the estimated depth in each direction at the time that the images were acquired from the wide-FOV cameras.

Note: Because the number of threads per block in CUDA is limited to 1024 and the number of pixels within a depth region is larger than that, each kernel thread must handle multiple rows. A shared-memory block is large enough for all pixels in the region, so it is used to provide special row-computing and column-computing threads access to data from each thread, and all threads can read at once to maximize initial memory bandwidth.

Note: Computing the coarse depth mesh on the GPU takes around 6ms for 7 candidate depths and two pairs of depth cameras. Computing the per-camera depth mesh from these estimates on a single CPU dominates the time, taking 17ms in the UpdateMesh() calls. Pushing depth meshes to the vertex buffers takes 1ms (and must be done in a single thread because it must have the OpenGL context). ***Attempting to CPU parallelize the per-camera depth mesh updates improves overall depth estimation time to around 11ms but causes dropped packets because it uses all CPUs and keeps the per-camera ingest threads from running often enough.***

Composite submodule



The diagram above, from *TR-006v29_Software_Architecture*, shows the operations performed by the Composite submodule. This submodule's operation is driven by methods provided for the Display submodule, one of which will be called to generate each viewpoint for each frame.

Because all pixels from all cameras are requested in the initial implementation, multiple Composite submodules can use the same ingested data. (When doing pilot-gaze-based calculations in the future, the Composite submodule will consider high-resolution foveal rendering insets vs. full-view rendering. One will be run for each Display submodule, requesting the appropriate subset streams.)

This submodule will render textures from the CUDA image buffers, mapping (and locking) the newest consistent set of buffers across all cameras during rendering. (A pool of four buffers is maintained for each camera, sorted in decreasing-time order. The buffer-selection approach depends on how we are running:

- **Live:** When a frame begins, the latest buffer from all cameras is checked to find the one that has a minimum time among all of them; the frame from each camera whose time most closely matches that time is chosen to provide frames from consistent time across all cameras.)
- **Replay:** Because the image generation cannot be synchronized to the display, an algorithm is adopted that works to ensure that the desired images have all arrived without increasing the required prediction interval. A single-frame offset is subtracted from the scan-out time, rendering as if 1 frame in the past. (This offset is also subtracted from the time sent to the PoseAdjuster so that we do not add to the duration of prediction beyond what would have been present during a live session.) The average of the previous frames used plus the frame time is compared to the desired offset from the scan-out time and used as the desired image time (unless this differs by more than 1 frame time from the offset from the scan-out time, in which case that offset time is used). The frame closest to this offset time (whether it is the first or second frame from each camera) is used, and then the average of the used frame times is stored for the next frame. This keeps the frame selection consistent so long as it does not drift too far from the desired time, handling both drift over time and the case where the desired time is right on the border between two times.

These buffers may have been adjusted by various optional submodules (NUC, Deblur, Tone Map).

The orientation, fields of view, and distortion correction for each camera will have been read from a calibration configuration file at start-up. For the initial implementation, these will be used to **construct distorted meshes in planes** that are at the same distance from each camera, and the closest point from all overlapping meshes will be rendered.

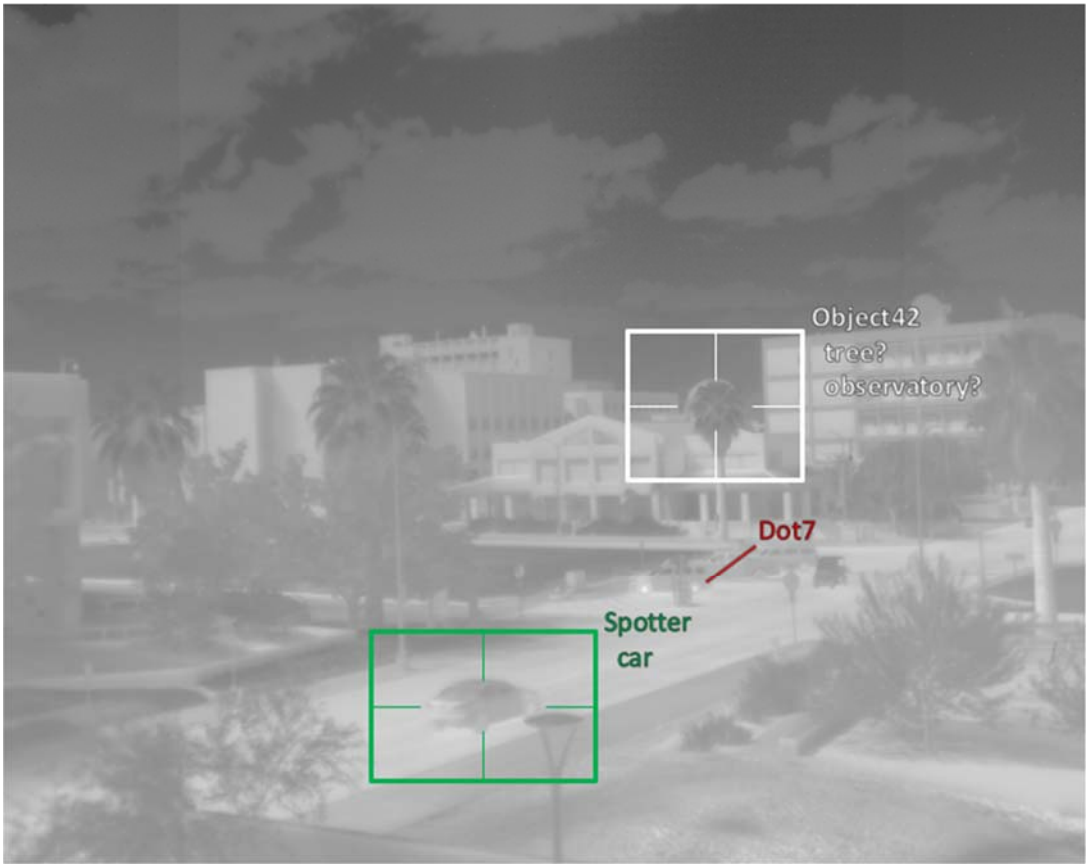
When the **optional Depth submodule** is available, the textures will be **projected onto the resulting manifold** rather than mapped onto distorted meshes. The distortion will be removed as part of the vertex shader, which will render each camera from its pose at the center of the image time. The image whose texture coordinate is closest to the center of the image will be used to color the pixel. If necessary for speed, only the four cameras whose principal rays pass closest to the region center will be tested for each region, and region sizes will be less than half the span of a camera image.

When the **optional Pose API** is available:

- Two poses will be estimated. The first is the interpolated pose of each image being used for this frame. The second is the predicted pose at the time of the center pixel in the frame being composited. The difference in poses will be used to **adjust the geometry** to be rendered onto or used for projection.
- Additionally, the rotational and translational velocity (when present) will be used to adjust the vertex shader for the distortion meshes. A differential rotation Quaternion and translation vector will be included in the parameters and used along with the Y coordinate to determine the fraction of these to apply to each vertex, shifting them to **compensate for motion during the frame**. This adjustment will not be made when reprojecting onto a depth manifold because it is solving an inverse projection problem.

When the **optional egocentric projection** is being used, the geometry is again moved to shift the origin by the inverse of the transformation from the camera center of projection to the center of the pilot's head at rest position.

When the optional **Analysis API** is being used as seen to the right, symbology and text are overlaid on the video data, displaying identified objects and their characteristics. These are drawn in response to the arrival of Analysis API JSON messages whose format is described in Appendix C TR-006v29.



The name of each object (which must be unique among all objects in a run) is displayed to the upper right of the geometric representation and is drawn as a screen-aligned billboard so that it is always readable. When one or more Class Type values are

specified, they are listed below the name – when there is more than one, they are listed in order of decreasing probability and have a "?" appended to each entry.

If a Class IFF is specified for the object, it is colored according to IFF: green for friend, red for foe, yellow for neutral and white for unknown or for an object that has more than one IFF.

When the object has no Rect entry, a line is drawn from a slight offset to the upper right. When it does have a Rec entry, an oversized rectangle is drawn that has four edge lines coming in as far as the actual size of the Rect. Both display choices aim to identify the object without drawing geometry over top of it that would obscure the object.

Semi-transparent dark halos are drawn around all geometry and text to provide contrast against an arbitrary background brightness and color, ensuring that the symbology is visible. These are drawn after the textured manifold so that they can mix properly with the background.

The same distortion correction and geometric registration pathway used to determine the locations of the textured manifold are used to determine the location and orientation for the lines, rectangles, and text. The points at all corners and edges are mapped to determine the location of each based on image pixel coordinates, and the Z distance is reduced slightly to ensure that they are drawn in front of the image textures.

When Vel data is available for an object, its timestamp and velocity are used to move the object to its expected location at the time of rendering, offsetting motion due to system detection and rendering latency.

A probability threshold can be set on the command line. Objects whose largest probability is below the threshold are not drawn. By default, this threshold is 0 so that all objects are drawn.

By default, when an object stops being reported it will continue to be displayed with its transparency fading linearly to 100% over a period of half a second. This provides persistence in the case of missed frames of identification for an object. During this time, the velocity information (if present) will be used to move the object along its expected trajectory. The duration of this persistence can be set using a command-line argument (setting it to 0 removes persistence).

Final composite: The resulting geometry will be rendered from the requested viewpoint into the buffer. The image manifold is drawn first, then the transparent halos, then the opaque lines and text.

Non-uniformity correction (NUC) submodules

The algorithms used for the IR and VIS NUC submodules are described in *TR-006v06_Software_Architecture*. Each will be implemented partially in pre-processing code (which determines the relative global gain and offset of each camera with respect to the others) and partially in the OpenGL shader as part of rendering (linearization, vignette, application of local and/or global offset and gain).

Histograms submodule

To support VIS cameras that have widely different exposure times and camera gains, the *Gain* metadata value is used to scale the top value of the histogram, which normally runs from 0-1. The number of bins will remain as specified, but the internally-stored maximum value will be adjusted. The histograms store floating-point normalized pixel counts per bin; the sum over all bins in a histogram is always 1. To support combining histograms, the number of pixels added to the histogram is stored separately within the object.

When combining two histograms, the range of the output histogram is the larger of the ranges of the two inputs. The total pixel count is the sum of the two inputs, and each set of input counts are scaled so that the resulting output is still normalized (with the histogram with more initial pixels weighted more heavily). When the bin sizes differ, the larger input bins are spread among the output bins proportionally for all bins covered.

Tone Map submodule (optional, may not be needed on VIS system)

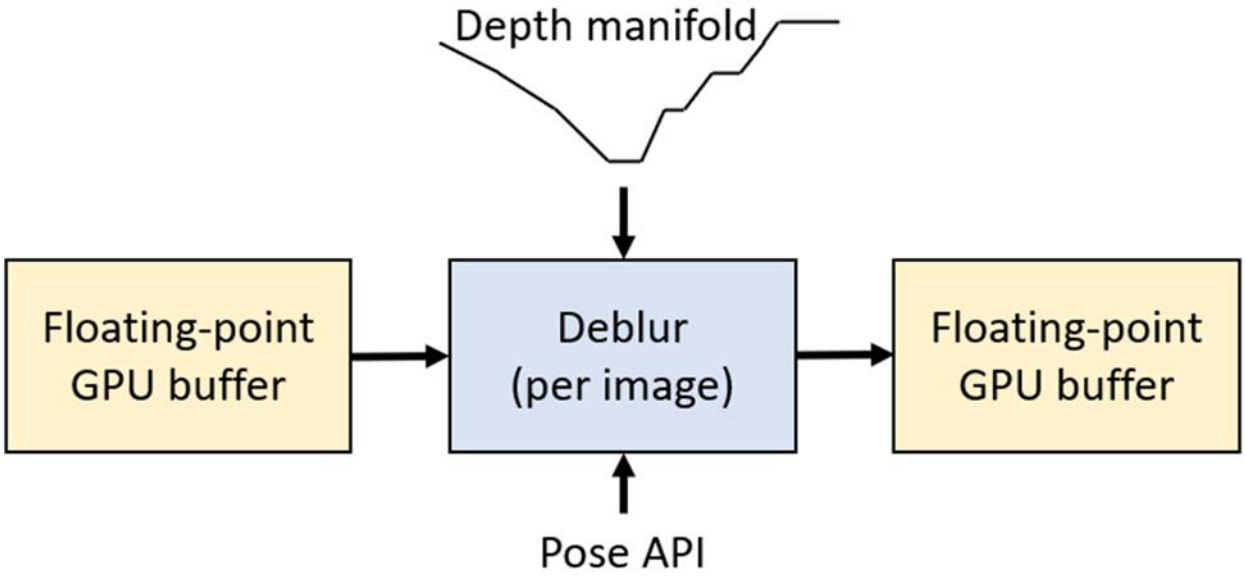
The Tone Map submodule is implemented in the OpenGL shader as part of rendering. It converts an incoming scaled and offset pixel brightness value into an RGB color to be displayed. Initially, R=G=B will be used for luminance-only display, but color maps with broader perceptual ranges will also be made available. Initially, gamma=1 will be used, but the submodule will support other gammas as well.

There are several proposed Tone Map generation algorithms, but the first to be implemented is one that has been shown to provide reasonable display for IR images. It proceeds as follows:

- **Mean+std:** The mean value and standard deviation of the histogram over all narrow-field images is computed.
- **Map:** The range from X standard deviations below the mean to Y standard deviations above is linearly mapped to the range 0-1.
- **Clamp:** The pixel values from all images are converted as above, clamping values to the range 0-1.

Others, including those described in *TR-006v08_Software_Architecture*, may also be implemented.

Deblur submodule (optional)



The Deblur submodule solves the inverse problem “What set of pixels is most likely to have produced this blurred output based on per-pixel estimated motion vectors?”. These motion vectors are computed based on the camera calibration information along with measured camera-rig motion. (The camera integration time to be considered is expected to be much larger than a single frame time, so progressive scan-out effects will be ignored in the initial implementation.)

The measured change in orientation at the center of the integration interval will be used to determine the local motion vector from each pixel; its image-based location change from half an interval before to half an interval after (the most-recent pixel read time) provides the estimated path of integration for the pixel.

When depth information is available, the depth manifold will be rendered from the point of view of the camera (including distortion correction in a second implementation). This will be used along with translational velocity to determine the translational path, which will be added to the rotational path to provide a complete per-pixel path estimate.

One or more “non-blind” (operating with a known kernel) deblurring techniques from the literature will be applied to determine an approach with sufficient speed and accuracy in the presence of IR image noise.

Appendix A: Direct-Display Approaches

To remove multiple frames of buffering from the operating system's compositor requires writing directly to a full-screen display that has been removed from the displays available for the window management system. This is known as writing to the displays in *Direct Mode*. There are both vendor-specific and general-purpose approaches to doing this, each of which is specific to an operating system. The following approaches are available. Unfortunately, providing shared OpenGL contexts between libraries is not possible, so a single library will be required for all threads in an application.

- **Cross-platform:**
  - **OpenXR Graphics Helper:** The OpenXR example programs have a helper library for graphics that supports the construction of full-screen, undecorated windows and the selection of which monitor they appear on. It supports vertical retrace synchronization.
  - **GLFW:**[9] This cross-platform (Windows, MacOS, X11, Wayland) library supports the construction of full-screen, undecorated windows and the selection of which monitor they appear on. It supports vertical retrace synchronization.
- **Windows 10 or 11:**
  - **OpenXR:**[10] The OpenXR runtime for an HMD can place it into Direct Mode.
  - **nVidia VRWorks:**[11] Provides vendor-specific access to direct displays on Windows. It also provides front-buffer rendering and context priority, enabling the display thread to have priority over all other GPU operations. This tool also provides a mechanism for causing a standard display to be treated as a Direct Mode display by the operating system, providing the ability to use a standard display with low latency.
  - **OSVR:**[12] The Open-Source Virtual Reality platform was developed to support monoscopic and stereo display on a variety of head-mounted displays. It has a DLL that supports Direct Mode rendering on nVidia, AMD, and Intel GPUs. It uses VRWorks on the nVidia platform and provides a common interface to all platforms along with the ability to do head tracking and post-rendering warp to minimize latency. (OSVR could be extended to provide Direct Mode on Linux by wrapping an available approach.)
  - **Windows.Devices.Display.Core API:**[13] Emables custom compositors for full-screen display, reducing latency when rendering to the display.[14] The documentation mentions the ability to manually switch a commercial display to Direct Mode using a UI, but there are not instructions for how to achieve this.
- **Linux:**
  - **Disable composition:**[15]
    - The GNOME[16] desktop environment uses "unredirection" for borderless full-screen windows, removing composition.
    - The LightDM[17] display manager provides a slimmed-down environment that can also display full-screen borderless windows.
    - **Don't run a Window Manager:** To avoid compositing done inside a desktop environment or window manager, avoid running one and place a single window within the application. This can be done by running *startx* without a window-manager argument or by using *xinit* directly.

[9] https://www.glfw.org
[10] https://www.khronos.org/openxr
[11] https://developer.nvidia.com/vrworks
[12] https://osvr.github.io
[13] https://learn.microsoft.com/en-us/uwp/api/windows.devices.display.core?view=winrt-22621
[14] https://learn.microsoft.com/en-us/windows-hardware/drivers/display/specialized-monitors-compositor
[15] https://linux-gaming.kwindu.eu/index.php?title=Compositor_(X11)
[16] https://www.gnome.org
[17] https://github.com/canonical/lightdm